

TRAVAUX DIRIGÉS — PYTHON

Les Dictionnaires en Python

Du dictionnaire simple à la combinaison liste–dictionnaire

Cours : Python — Dictionnaires	Enseignant : Mouhamad DJIGO
Nombre d'exercices : 20	

Objectifs pédagogiques

- Créer, accéder et modifier des dictionnaires Python
- Parcourir un dictionnaire avec des boucles
- Combiner dictionnaires et listes pour résoudre des problèmes complexes
- Utiliser les méthodes : `get()`, `keys()`, `values()`, `items()`, `len()`, `del`, `in`

Partie I — Création et affichage de dictionnaires

Exercice 1. Mon premier dictionnaire

Un étudiant souhaite enregistrer ses informations personnelles. Écrire un programme qui crée un dictionnaire contenant les clés : nom, prenom, age, ville et filiere. Le programme affiche ensuite chaque information avec un message descriptif.

Exemple de sortie :

```
Nom : Moussa
Prenom : Diallo
Age : 20
Ville : Dakar
Filiere : Informatique
```

Contraintes :

- Ne pas utiliser de liste.
- Toutes les valeurs doivent être saisies par l'utilisateur via `input()`.
- Afficher chaque valeur avec `print()` et en passant par leur cle.

Exercice 2. Dictionnaire d'un produit

Un commerçant veut enregistrer les informations d'un produit en stock. Écrire un programme qui crée un dictionnaire avec les clés : nom, prix, quantite et categorie. Le programme affiche le dictionnaire complet.

Exemple de sortie :

```
{'nom': 'Téléphone', 'prix': 85000, 'quantite': 10, 'categorie':  
'Électronique'}
```

Contraintes :

- Pour cet exercice uniquement, les valeurs peuvent être statiques (non saisies).
- Utiliser des accolades {} pour déclarer le dictionnaire.
- Ne pas utiliser de liste.

Exercice 3. Dictionnaire construit clé par clé

Un agent de santé veut créer une fiche patient. Écrire un programme qui commence par un dictionnaire vide, puis ajoute une à une les clés suivantes saisies par l'utilisateur : nom, age, groupe_sanguin, poids et taille. Le programme affiche la fiche complète.

Exemple de sortie :

```
--- Fiche Patient ---  
nom           : Awa  
age           : 25  
groupe_sanguin: O+  
poids         : 62  
taille        : 165
```

Contraintes :

- Commencer avec un dictionnaire vide : `fiche = {}`
- Ajouter chaque clé une à une avec `fiche['cle'] = valeur`.
- Ne pas utiliser de liste.
- Afficher les données avec une boucle `for` et `items()`.

Partie II — Accès, modification et suppression

Exercice 4. Accès aux valeurs d'un dictionnaire

Un vendeur souhaite consulter les informations d'un article. Écrire un programme qui crée un dictionnaire article avec les clés nom, prix, marque et stock (valeurs saisies par l'utilisateur). Le programme demande ensuite à l'utilisateur quelle clé il souhaite consulter et affiche la valeur correspondante. Si la clé n'existe pas, un message d'erreur est affiché.

Exemple de sortie :

```
Quelle information souhaitez-vous consulter ? prix  
Valeur : 45000
```

Contraintes :

- Utiliser la méthode `get()` pour accéder à la valeur.
- Ne pas utiliser de `if/else` pour vérifier manuellement chaque clé.
- Afficher 'Clé non trouvée.' si la clé n'existe pas, via la valeur par défaut de `get()`.

Exercice 5. Mise à jour du profil d'un utilisateur

Un réseau social permet à ses utilisateurs de modifier leur profil. Écrire un programme qui crée un dictionnaire utilisateur avec les clés `pseudo`, `email`, `age` et `ville` (valeurs saisies). Le programme demande ensuite quelle clé modifier et la nouvelle valeur, puis affiche le profil mis à jour.

Exemple de sortie :

```
Profil mis à jour :  
pseudo : AlphaDev  
email  : alpha@mail.com  
age    : 22  
ville  : Thiès
```

Contraintes :

- Ne pas réécrire tout le dictionnaire pour modifier une clé.
- Utiliser `dico['cle'] = nouvelle_valeur` pour la mise à jour.
- Vérifier que la clé existe avec `in` avant de modifier.

Exercice 6. Suppression d'entrées dans un annuaire

Un secrétaire gère un annuaire téléphonique. Écrire un programme qui crée un dictionnaire de 4 contacts (`nom` → `numéro`), avec les noms et numéros saisis par l'utilisateur. Le programme demande ensuite quel contact supprimer, le supprime et affiche l'annuaire mis à jour.

Exemple de sortie :

```
Contact 'Omar' supprimé.  
Annuaire :  
Fatou : 771234567  
Ibou  : 785654321  
Awa   : 763112233
```

Contraintes :

- Utiliser `del` pour supprimer la clé.
- Vérifier que le contact existe avec `in` avant de supprimer.
- Afficher un message d'erreur si le contact est introuvable.

Partie III — Parcours de dictionnaires

Exercice 7. Bulletin de notes

Un professeur veut afficher le bulletin de notes d'un étudiant. Écrire un programme qui saisit le nombre de matières, puis pour chaque matière saisit le nom et la note. Le programme affiche un bulletin formaté.

Exemple de sortie :

```
--- Bulletin de notes ---  
Mathématiques   : 15/20  
Physique        : 12/20  
Informatique    : 18/20
```

Contraintes :

- Utiliser une boucle for pour saisir les matières.
- Parcourir le dictionnaire avec items() pour l'affichage.
- Ne pas utiliser de liste séparée pour les matières.

Exercice 8. Inventaire d'un magasin

Un gérant de magasin veut connaître la valeur totale de son inventaire. Écrire un programme qui saisit le nombre d'articles, puis pour chaque article saisit le nom, le prix unitaire et la quantité. Le programme calcule et affiche la valeur totale de chaque article et le total général.

Exemple de sortie :

```
Ordinateur : 350000 x 5 = 1750000 FCFA  
Souris     : 15000  x 12 = 180000 FCFA  
Total général : 1930000 FCFA
```

Contraintes :

- Stocker les articles dans un dictionnaire : nom → {'prix': ..., 'quantite': ...}.
- Utiliser items() pour parcourir et calculer.
- Ne pas utiliser de liste séparée.

Exercice 9. Moyenne générale depuis un dictionnaire

Un délégué de classe collecte les moyennes de ses camarades. Écrire un programme qui saisit le nombre d'étudiants, puis pour chaque étudiant saisit le nom et la moyenne. Le programme affiche tous les étudiants avec leur moyenne, puis calcule et affiche la moyenne générale de la classe.

Exemple de sortie :

```
Moussa : 14.5  
Fatou  : 16.0  
Cheikh : 12.5  
Moyenne générale : 14.33
```

Contraintes :

- Utiliser `values()` pour calculer la moyenne.
- Ne pas utiliser de liste séparée pour les moyennes.
- Utiliser `len()` pour compter le nombre d'étudiants.

Partie IV — Conditions et recherches dans un dictionnaire

Exercice 10. Attribution des mentions

Un enseignant veut attribuer automatiquement des mentions à ses étudiants. Écrire un programme qui saisit le nombre d'étudiants, puis pour chaque étudiant saisit le nom et la moyenne. Le programme crée un dictionnaire `nom → mention` et affiche le résultat. Les mentions : ≥ 16 Très bien, ≥ 14 Bien, ≥ 10 Assez bien, sinon Insuffisant.

Exemple de sortie :

```
Ali      : Très bien
Ndeye    : Bien
Oumar    : Assez bien
```

Contraintes :

- Utiliser un seul dictionnaire `nom → mention`.
- Ne pas utiliser de liste.
- Les conditions `if/elif/else` doivent être imbriquées dans la boucle de saisie.

Exercice 11. Recherche dans un dictionnaire de capitales

Un quiz de géographie demande à l'utilisateur de deviner des capitales. Écrire un programme qui crée un dictionnaire de 5 pays/capitales (valeurs saisies). Ensuite, le programme pose des questions à l'utilisateur en boucle : il saisit un pays et doit deviner la capitale. Le programme indique si la réponse est juste ou fausse. La saisie s'arrête quand l'utilisateur tape 'fin'.

Exemple de sortie :

```
Entrez un pays : Sénégal
Capitale devinée : Dakar
Bonne réponse !
Entrez un pays : fin
```

Contraintes :

- Utiliser `get()` pour chercher la capitale.
- Utiliser une boucle `while True` avec `break`.
- Ne pas utiliser de liste.

Partie V — Dictionnaires et boucles avec conditions d'arrêt

Exercice 13. Enregistrement de ventes avec arrêt

Un commerçant saisit ses ventes au fur et à mesure de la journée dans une liste. Écrire un programme qui saisit successivement le nom du produit vendu et le montant de chaque vente. Le programme calcule et affiche le total des ventes et le produit le plus vendu (en montant). La saisie s'arrête quand l'utilisateur saisit 'fin' comme nom de produit.

Exemple de sortie :

```
Total des ventes : 275000 FCFA
Produit le plus vendu : Téléphone (150000 FCFA)
```

Contraintes :

- Utiliser une boucle while True avec break.
- Stocker les ventes dans un dictionnaire produit → montant.

Exercice 14. Registre de présences

Un enseignant prend les présences de sa classe. Écrire un programme qui saisit successivement le nom d'un étudiant et sa présence (1 = présent, 0 = absent). Le programme affiche le nombre de présents, le nombre d'absents et la liste des absents. La saisie s'arrête quand l'utilisateur saisit 'fin' comme nom.

Exemple de sortie :

```
Présents : 3
Absents : 2
Liste des absents : Omar, Awa
```

Contraintes :

- Utiliser une boucle while True avec break.
- Stocker les présences dans un dictionnaire nom → 1/0.
- Utiliser values() pour compter présents et absents.
- Ne pas utiliser de liste séparée.

Exercice 15. Gestion d'un stock avec alertes

Un responsable de stock saisit les quantités disponibles pour N produit. Écrire un programme qui saisit le nom et la quantité de chaque produit. Le programme affiche une alerte pour tout produit dont la quantité est inférieure à un seuil saisi au départ.

Exemple de sortie :

```
⚠ ALERTE : Sucre — quantité faible (3 restants)
⚠ ALERTE : Huile — quantité faible (1 restants)
```

Contraintes :

- Stocker les produits dans un dictionnaire nom → quantité.
- Parcourir avec `items()` pour détecter les alertes.
- Ne pas utiliser de liste.

Partie VI — Structures avancées : dictionnaires imbriqués

Exercice 16. Fiche étudiante complète

Un secrétaire pédagogique veut créer une fiche complète pour chaque étudiant. Écrire un programme qui saisit le nom d'un étudiant et ses notes dans `n` matières. Le programme stocke les données dans un dictionnaire contenant le nom et une sous-liste de notes. Il calcule et affiche la moyenne de l'étudiant.

Exemple de sortie :

```
Étudiant : Aminata
Notes : [14, 16, 12, 18]
Moyenne : 15.0
```

Contraintes :

- Utiliser un dictionnaire avec une clé 'notes' contenant une liste.
- Ne pas calculer la moyenne avec une boucle — utiliser `sum()` et `len()`.
- Afficher les informations avec un accès par clé, pas par indice.

Exercice 17. Annuaire multi-informations

Un club sportif veut gérer un annuaire de ses membres. Écrire un programme qui saisit le nombre de membres (liste de membre). Pour chaque membre, on saisit : nom, sport pratiqué, numéro de téléphone et âge. Les données sont stockées dans un dictionnaire de dictionnaires. Le programme affiche ensuite la fiche de tous les membres.

Exemple de sortie :

```
--- Annuaire du club ---

Nom : Moussa
Sport      : Football
Téléphone  : 771234567
Âge       : 22
```

Contraintes :

- Utiliser une liste principal dont chaque valeur est un dictionnaire.
- Parcourir avec `items()` pour l'affichage.

Partie VII — Listes de dictionnaires

Exercice 18. Classement d'étudiants

Un professeur veut classer ses étudiants par ordre de moyenne décroissante. Écrire un programme qui saisit le nombre d'étudiants puis, pour chacun, le nom et la moyenne. Les données sont stockées dans une liste de dictionnaires. Le programme affiche le classement du meilleur au moins bon.

Exemple de sortie :

```
--- Classement ---
1. Ndeye   : 17.5
2. Ali     : 15.0
3. Oumar   : 12.0
```

Contraintes :

- Utiliser une liste de dictionnaires : `[{'nom': ..., 'moyenne': ...}, ...]`.

Exercice 19. Gestion d'une bibliothèque

Un bibliothécaire veut gérer les livres disponibles. Écrire un programme qui saisit le nombre de livres. Pour chaque livre, on saisit : titre, auteur, année et disponibilité (1 = disponible, 0 = emprunté). Le programme affiche la liste des livres disponibles et le nombre total de livres empruntés.

Exemple de sortie :

```
Livres disponibles :
- Python pour tous (2021)
- Algorithmique (2019)
Livres empruntés : 2
```

Contraintes :

- Stocker les livres dans une liste de dictionnaires.
- Filtrer les disponibles avec une boucle `for` et une condition sur la clé 'disponible'.
- Ne pas utiliser `filter()` ou `list comprehension`.

Partie VIII — Exercice de synthèse

Exercice 20. Système de gestion de classe

Un délégué de classe veut créer un mini-système de gestion. Écrire un programme qui saisit les informations de n étudiants : nom, âge et notes dans 3 matières (Maths, Physique, Informatique). Pour chaque étudiant, le programme calcule la moyenne et attribue une mention. À la fin, il affiche : la liste complète des étudiants avec toutes leurs infos, l'étudiant ayant la meilleure moyenne, et la moyenne générale de la classe.

Exemple de sortie :

```
--- Liste des étudiants ---  
Fatou | Âge: 20 | Moy: 15.67 | Mention: Bien ...  
Meilleur étudiant : Fatou (15.67)  
Moyenne de la classe : 13.44
```

Contraintes :

- Utiliser une liste de dictionnaires pour stocker les étudiants.
- Chaque dictionnaire doit contenir : nom, âge, notes (dict matière→note), moyenne, mention.
- Calculer la moyenne avec `sum()` et `len()` sur les valeurs du dictionnaire de notes.